

1. Why do we use IActionResult instead of ActionResult?

`IActionResult` is an interface that allows an action method to return different types of HTTP responses, such as `Ok()`, `NotFound()`, and `Redirect()`.

Example:

```
public IActionResult GetUser(int id)
{
    if (id <= 0)
        return BadRequest();

    return Ok("User found");
}
```

Why? It gives us flexibility to return different responses depending on the situation.

`ActionResult` is an abstract class that can also be used as a return type, while `ActionResult<T>` is useful when returning a specific data type or an HTTP response.

2. What do HTTP Request and Response Messages Consist of?

HTTP Request consists of:

- **Request Line:** HTTP method, URL, and HTTP version.
- **Headers:** Additional information such as content type and authorization.
- **Body:** Data sent to the server, usually with POST or PUT requests.

Example:

```
POST /api/users HTTP/1.1
Content-Type: application/json
```

```
{"name": "Ahmed"}
```

HTTP Response consists of:

- **Status Line:** HTTP version, status code, and status message.
- **Headers:** Information about the response, such as content type.
- **Body:** The returned data or error message.

Example:

```
HTTP/1.1 200 OK  
Content-Type: application/json
```

```
{"message": "Success"}
```

3. What Is the Difference Between HTTP and HTTPS?

- **HTTP:** Transfers data between the client and server without built-in encryption.
- **HTTPS:** Uses TLS encryption to protect data during transmission.

Example: When logging into a banking website, HTTPS protects sensitive information such as passwords and financial data from being easily intercepted.

HTTPS provides confidentiality, data integrity, and server authentication.

4. What Are URL Segments and Fragments?

A URL can contain several parts, including the path, query string, and fragment.

Example:

<https://example.com/products/25?category=books#reviews>

- **Scheme:** https
- **Host:** example.com
- **Path segments:** products and 25
- **Query string:** ?category=books — passes additional parameters.
- **Fragment:** #reviews — identifies a specific section of the page.

In this example, 25 could represent a product ID, while #reviews could direct the browser to the reviews section.

5. What Are Builder and Dependency Injection?

Builder Pattern: A design pattern used to create and configure complex objects step by step.

Example: In ASP.NET Core, `WebApplication.CreateBuilder(args)` creates a builder that allows us to configure services, logging, and application settings before building the application.

Dependency Injection (DI): A technique where a class receives the objects it depends on instead of creating them itself.

Real-life example: A car needs an engine. Instead of creating the engine inside the car class, we inject an engine into it.

```
public class Car
{
    private readonly IEngine _engine;

    public Car(IEngine engine)
    {
        _engine = engine;
    }
}
```

In ASP.NET Core, DI improves maintainability, testability, and loose coupling.

6. What Is the Difference Between Web Pages (Razor Pages) and MVC?

Razor Pages:

- Page-focused architecture.
- Each page usually has a `.cshtml` file and a corresponding `PageModel`.
- Best for applications where functionality is organized around individual pages.

MVC (Model-View-Controller):

- Separates the application into Model, View, and Controller.
- Controllers handle requests, Models represent data and business logic, and Views display the UI.
- Best for applications that benefit from centralized request handling and clear separation of responsibilities.

Business Case 1 — Company Website:

Razor Pages can be suitable for a company website with Home, About, Contact, and FAQ pages because each page has its own logic.

Business Case 2 — E-commerce System:

MVC can be suitable for a large e-commerce system with products, orders, customers, and payments because it provides a clear separation between controllers, models, and views.

Comparison: Both support ASP.NET Core, routing, and Razor syntax. Razor Pages are page-focused, while MVC separates responsibilities using controllers, models, and views.

7. What Is Content-Type in an HTTP Response?

Content-Type is an HTTP header that tells the client what type of data the response body contains.

Examples:

Content-Type: application/json

Used when returning JSON data, commonly in Web APIs.

Content-Type: text/html

Used when returning HTML pages.

Content-Type: image/png

Used when returning PNG images.

Why do we use it? It helps the browser or client understand how to interpret and process the response body correctly.

8. What Are Minification, Web Bundling, Webpack, and Lazy Loading?

Minification: Removes unnecessary characters, whitespace, and comments from CSS and JavaScript files without changing their functionality. This reduces file size.

Web Bundling: Combines multiple files into one or fewer files to reduce the number of network requests.

Webpack: A JavaScript module bundler that combines modules and can optimize application assets through bundling, minification, and code splitting.

Lazy Loading: Loads code or resources only when they are needed instead of loading everything at the beginning.

Example: In an e-commerce website, the checkout code can be loaded only when the user opens the checkout page.

How do they improve performance?

- Minification reduces the amount of data transferred.
- Bundling can reduce the number of network requests.
- Webpack helps organize and optimize assets.
- Lazy loading reduces the initial amount of code that must be downloaded and processed.

Together, these techniques can improve loading speed and reduce unnecessary network traffic.